



Git & GitHub

— H A N D B O O K —

From Beginner to Job Ready



CODE



BRANCH



COLLABORATE

By Prathvi Singh Thakur

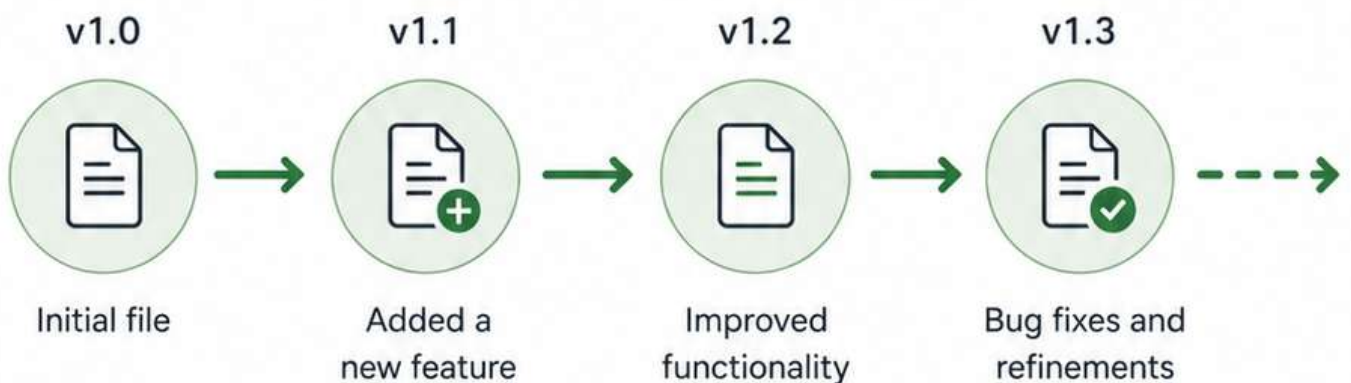
What is Version Control?

Version control is a system that records changes to files over time so you can recall specific versions later, collaborate with others, and work more efficiently.



Version control helps track changes, collaborate, and manage code efficiently.

How Versions Evolve



If something goes wrong, you can go back to any previous version. If multiple people are working, version control helps merge changes smoothly without overwriting each other.

Git vs GitHub

Understand the Difference



Git (Tool)

Version Control System



Version Control

Tracks changes in your files over time.



Local System

Works on your local machine.



Command Line Tool

Operated using Git commands.



Distributed System

Full history is available locally.



Fast & Lightweight

Designed for speed and efficiency.

VS



GitHub (Platform)

Cloud-based Hosting Service



Cloud Hosting

Hosts your repositories in the cloud.



Collaboration

Enables teams to work together easily.



Remote Repositories

Store and manage code remotely.



Additional Features

PRs, Issues, Wiki, Actions, Projects and more.



Access Anywhere

Access your code from anywhere in the world.



In Simple Words

Git is the tool you use to track changes in your code.

GitHub is the platform you use to store your code online and collaborate with others.

Installing & Configuring Git

Follow these simple steps to install Git and set up your identity.



1. Install Git

Download and install Git from the official website:

 <https://git-scm.com/downloads>



2. Configure Your Identity

Set your name and email. These will be used in your commits.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```



3. Verify Configuration

Check that your settings are correct.

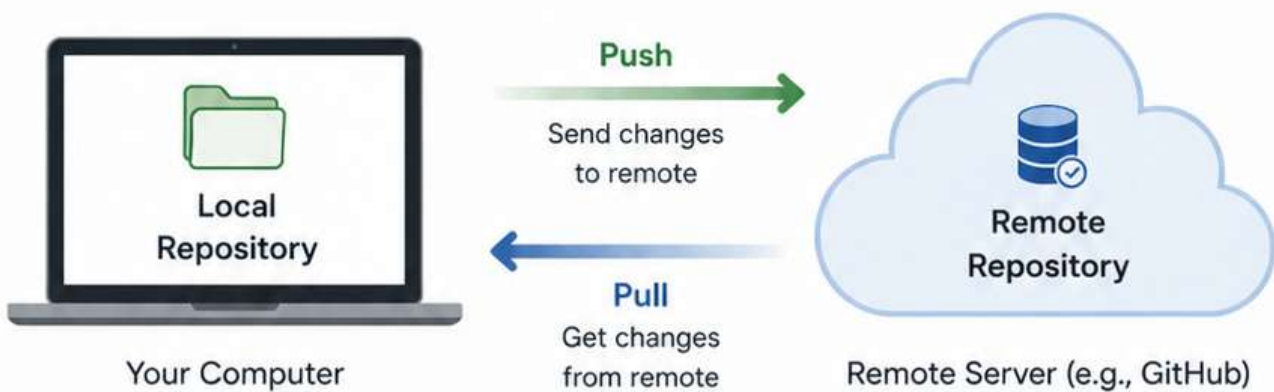
```
Terminal
$ git config --global user.name
Your Name
$ git config --global user.email
your@email.com
```



Tip: Use `--global` to apply these settings to all your repositories.

Local vs Remote Repository

In Git, your work happens in two places: your local machine and a remote server. They work together to keep your code safe and up to date.



Local Repository

- Exists on your computer.
- You can work offline.
- All changes and history are stored locally.
- You have full control.



Remote Repository

- Stored on a remote server (e.g., GitHub).
- Accessible from anywhere.
- Used for backup and collaboration.
- Keeps your code safe and in sync with others.



Tip: You commit changes in your local repository and push them to the remote repository to share or back them up.

Essential Git Commands

Common commands you'll use every day



1

git init

Initialize a new Git repository.



2

git clone

Clone an existing repository.



3

git add

Stage changes to be committed.



4

git commit

Commit staged changes to the repository.



5

git status

Show the current status of changes.



6

git log

View the commit history of the repository.



Tip: These commands form the foundation of Git. Practice them regularly to become more confident!



Your Goal

Track changes, create commits,
and manage code **confidently**.



Every commit is progress.

Keep learning, keep building,
and grow with every step.



You've got this!



Git Branches

Branches let you work on features or fixes independently without affecting the main codebase.

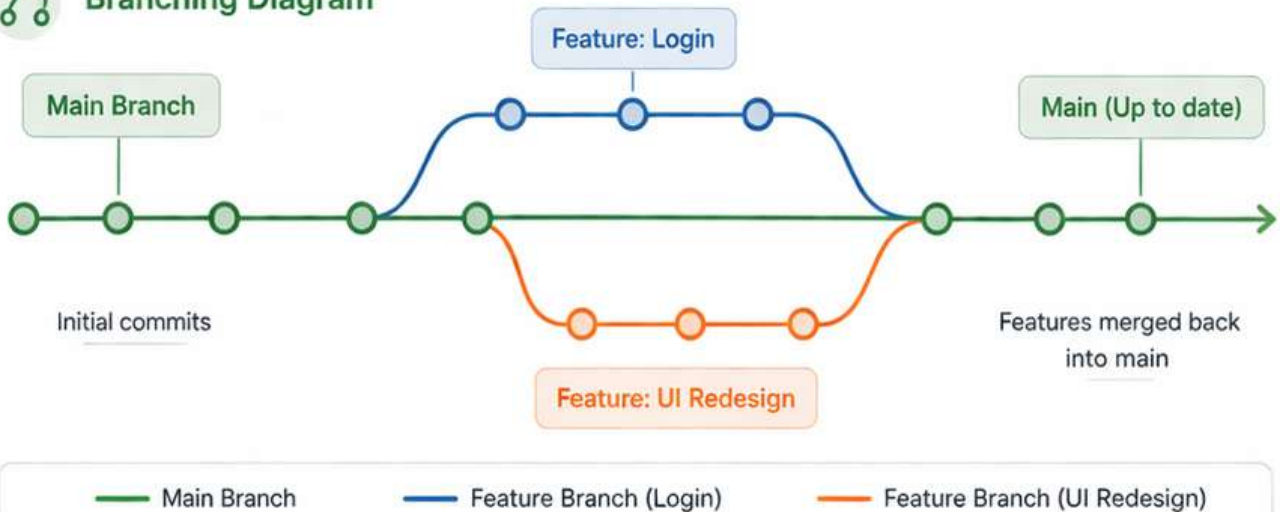


What is a Branch?

A branch is a separate line of development. It allows you to work on new features, fix bugs, or experiment safely.



Branching Diagram



Useful Commands



`git branch`

List, create, or delete branches.

```
# List branches
git branch
# Create a new branch
git branch feature/login
# Delete a branch
git branch -d feature/login
```



`git checkout`

Switch to a branch or restore files.

```
# Switch to an existing branch
git checkout feature/login
# Create and switch
git checkout -b feature/login
```



`git switch`

Switch to a branch (modern alternative).

```
# Switch to an existing branch
git switch feature/login
# Create and switch
git switch -c feature/login
```



Tip: Work in branches, test safely, and merge with confidence. It keeps your main branch clean and stable.

Merge Types

Git provides different ways to bring changes together.

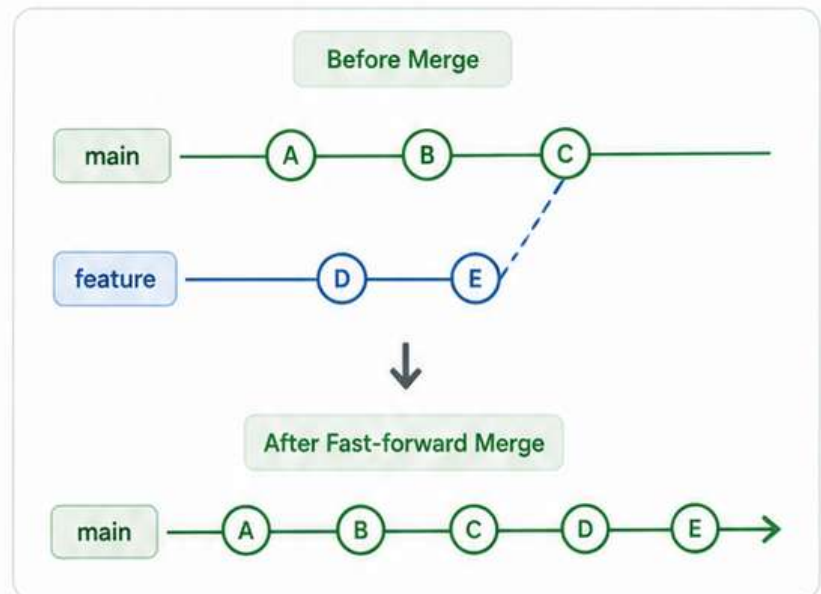
1 Fast-forward Merge

When the target branch has no new commits, Git simply moves it forward to the tip of the source branch.



Key Point

History stays linear and clean.



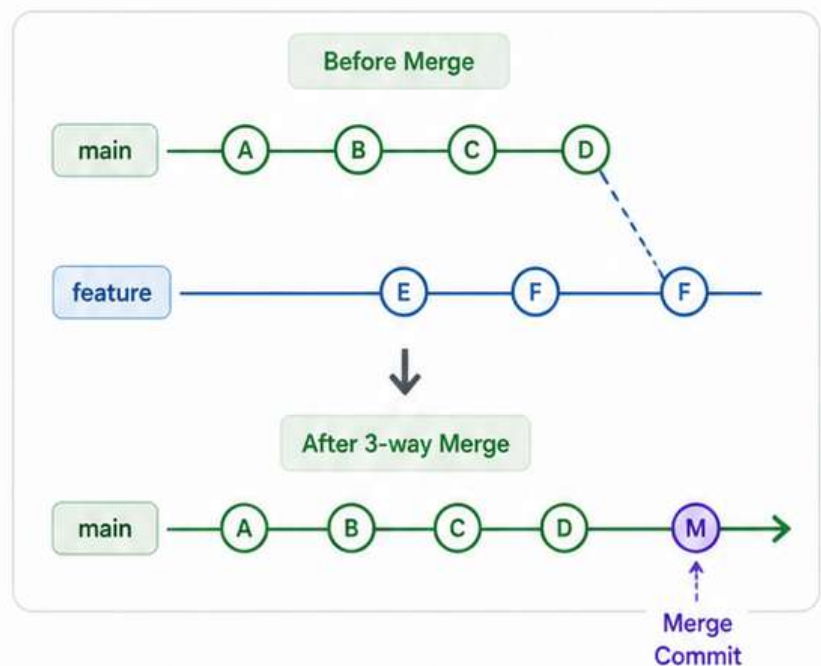
2 3-way Merge

When the target branch has new commits, Git creates a merge commit (M) to combine both histories.



Key Point

A merge commit (M) keeps both histories and shows where they were combined.



Tip: Fast-forward keeps history simple.
3-way merge preserves the complete story.

Merge Conflicts

A merge conflict happens when Git can't automatically resolve differences between changes. You'll need to resolve them manually.



Why Conflicts Happen

When two branches modify the same part of a file differently, Git doesn't know which change to keep. It marks the conflict so you can decide.



Example: Conflict in a File

```
1 function greet() {
2 <<<<<<< HEAD (current branch)
3   console.log("Hello, World!");
4 =====
5   console.log("Hello, Git!");
6 >>>>>> feature-branch (incoming branch)
7 }
```

Changes from
your branch

Separator

Changes from
the other branch



How to Resolve Merge Conflicts

1



Open the file

Git will show the files with conflicts.

2



Edit the file

Choose the correct changes or combine them as needed. Remove the conflict markers.

3



Save the file

Save your changes after resolving the conflict.

4



Stage the file

Mark the resolved file as ready to commit.

```
git add <file>
```

5



Commit the merge

Complete the merge with a commit.

```
git commit
```



Tips

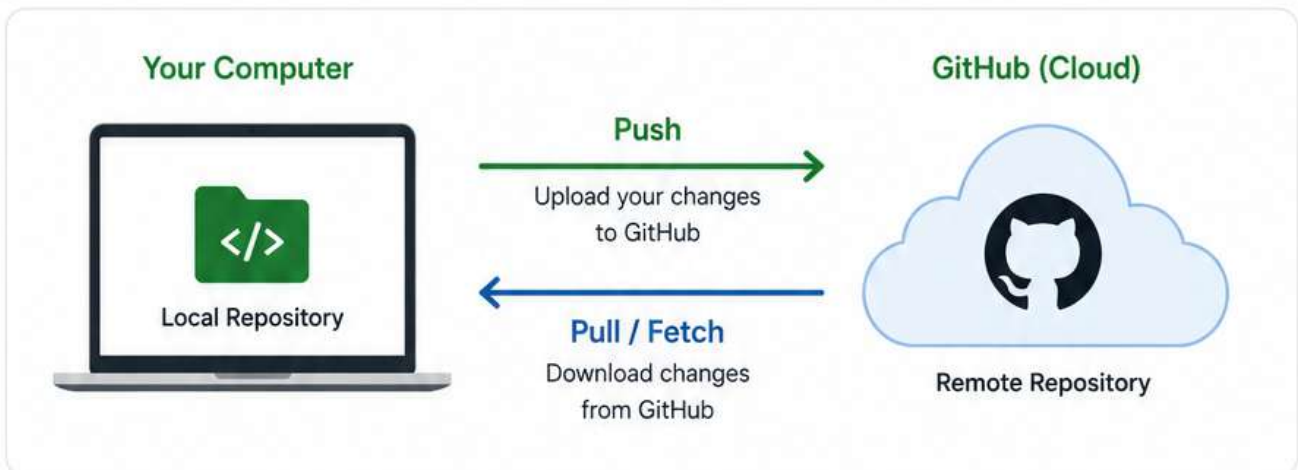
- Take your time and review both changes carefully.
- Test your code after resolving conflicts.
- Communicate with your team to reduce conflicts.



Conflicts are normal. Resolving them makes you a better developer.

Introducing GitHub

GitHub is a cloud platform that hosts your Git repositories, enables collaboration, and keeps your code safe.



★ Key Actions



1 Create a Repository

Create a new repository on GitHub to store your project.

Option A: On GitHub Website

Go to github.com → New repository → Add name → Create.



2 Push

Upload your local commits to GitHub.

```
$ git push origin main
# Pushes your local changes to the
remote repository
```



3 Pull

Download and merge changes from GitHub to your local branch.

```
$ git pull origin main
# Fetches and merges changes from the
remote repository
```



4 Fetch

Download changes from GitHub without merging them.

```
$ git fetch origin
# Fetches changes. You can review and
merge manually.
```



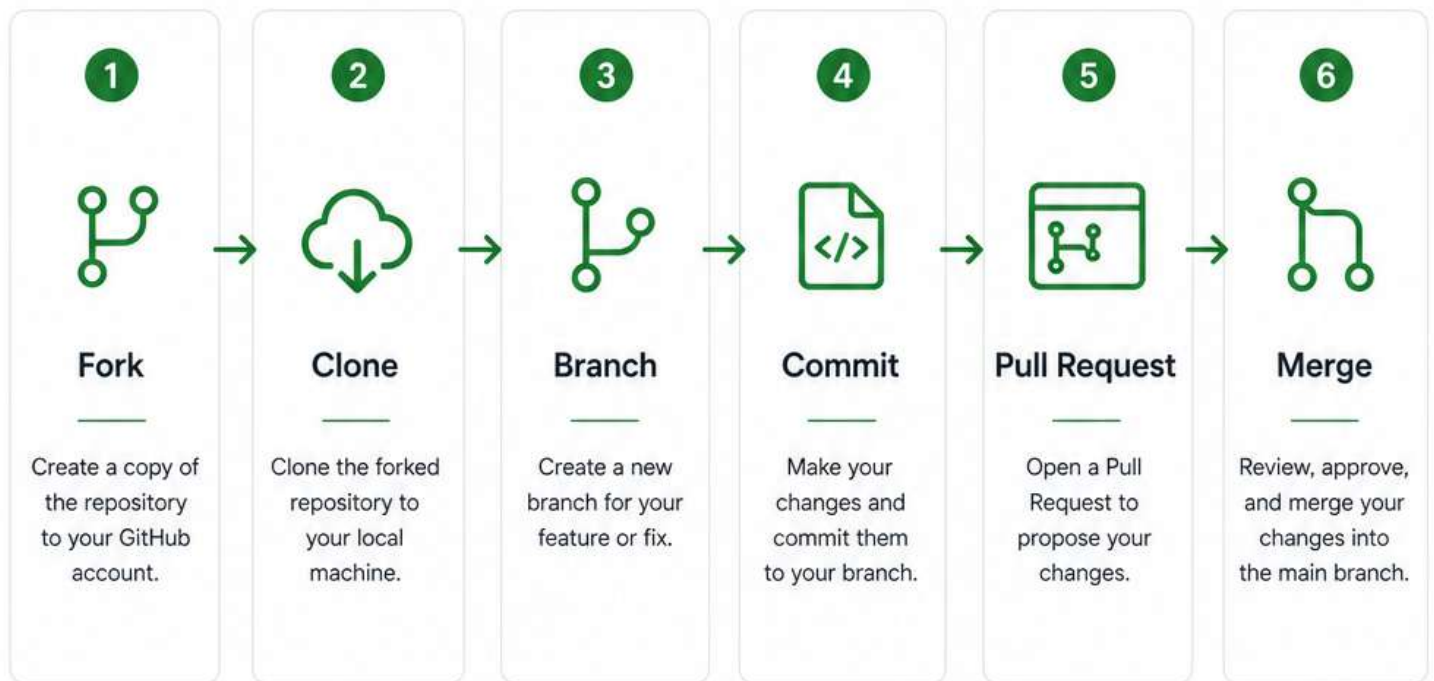
Why it matters?

GitHub keeps your code backed up, makes collaboration easy, and lets you work from anywhere.



The GitHub Flow

Contribute to a project using a simple, common workflow.



Why this flow?

- Keeps the main branch stable
- Encourages collaboration and code review
- Makes your contributions clear and organized



Tip: Keep your branches up to date and write clear commit messages.

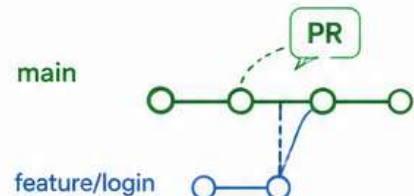
Pull Requests

Pull Requests (PRs) let you propose changes to a repository and collaborate with others before merging.



What is a Pull Request?

A Pull Request is a way to show your changes, request feedback, and merge them into the main branch.



The Pull Request Workflow

1

Create PR

Open a Pull Request to propose your changes.

**2**

Review Code

Team members review, comment, and suggest improvements.

**3**

Merge Changes

Once approved, your changes are merged into the main branch.



Pull Request in GitHub (Example)

Add login validation #42

[Edit](#)[New review](#)[Open](#)[feature/login](#)[main](#)

Conversation 3



Commits 4



Checks 2



Files changed 5



dev-user commented 2 hours ago

This PR adds validation for login form and improves error messages.



dev-user added 4 commits 1 day ago



All checks have passed 1 hour ago

Reviewers



alice



bob



Labels

[enhancement](#)

Milestone

v1.2



Why Pull Requests?

- Encourage collaboration and knowledge sharing
- Catch bugs early through review
- Keep the main branch stable and high quality



Undoing Changes

git reset vs git revert vs git checkout

Different commands for different situations.

Feature	 git reset	 git revert	 git checkout
 Purpose	Move the current branch pointer to a different commit. Discard changes.	Create a new commit that undoes the changes of a specific commit. Preserve history.	Switch branches or restore working directory files from a commit. Does not undo commits.
 Affects History	Yes Rewrites history (use with caution)	No Adds a new commit (safe for shared history)	No Only moves HEAD or restores files
 When to Use	- Local commits not pushed yet - Clean up mistakes - Rewrite history before sharing	- Undo changes already pushed - Safe for shared repositories - Maintain a clear history	- Switch branches - Restore a file or folder from a commit - Explore code or old versions
 Example Command	<pre>git reset --hard HEAD~1 # Move back 1 commit and discard changes</pre>	<pre>git revert <commit-hash> # Create a new commit that undoes the changes</pre>	<pre>git checkout <branch> # Switch to a branch git checkout <file> # Restore file from current commit</pre>
 Effect on Working Directory	--soft: keeps changes --mixed (default): keeps changes unstaged --hard: discards changes	No change to working directory (just a new commit added).	Files may change when switching branches or restoring from a commit.
 Safety	Risky Can lose work if used incorrectly.	Safe Does not remove history or delete commits.	Safe Does not change history.



Key Takeaway

Use **git reset** to rewrite history (locally). Use **git revert** to undo changes safely (publicly). Use **git checkout** to switch context or restore files.

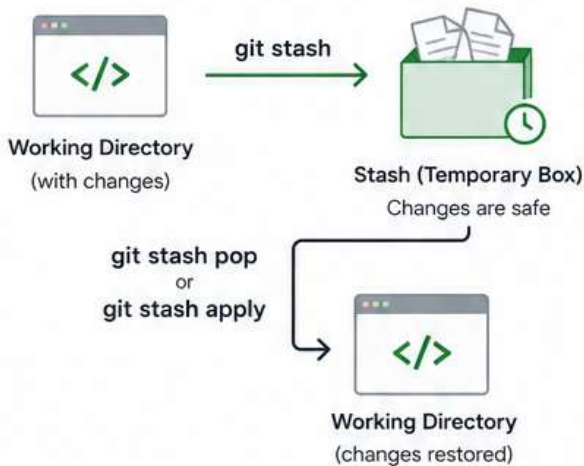
git stash & git clean

Keep your working directory clean and focused.

git stash

Temporarily save your changes (tracked and staged) so you can switch branches or work on something else.

Stash = Temporary Storage



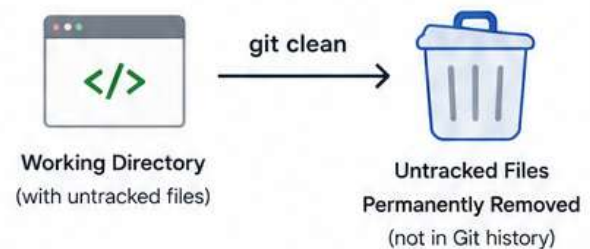
Common Usage

<code>git stash</code>	Stash your changes
<code>git stash list</code>	List all stashes
<code>git stash pop</code>	Apply latest stash and remove it
<code>git stash apply</code>	Apply latest stash (keep it)
<code>git stash drop</code>	Delete a stash

git clean

Remove untracked files and directories from your working directory.
These files are not stored by Git.

Clean = Remove Untracked Files



Common Usage

<code>git clean -n</code>	Preview what will be removed (dry run)
<code>git clean -f</code>	Remove untracked files
<code>git clean -fd</code>	Remove untracked files and directories

Key Differences

Feature	git stash	git clean
What it handles	Tracked & staged changes	Untracked files & directories
Where it goes	Saved in stash (temporary storage)	Permanently deleted
Can restore?	Yes, you can apply/pop it back	No, cannot be recovered easily
Affects Git history?	No	No
Safe to use?	Yes, changes are safe	Use carefully! Deleted files are gone



Pro Tip

Use `git stash` when you want to pause your work.
Use `git clean` when you want a truly clean workspace.



Use the right tool for the job. Stay clean. Stay productive.

git rebase vs git merge

Two ways to integrate changes from one branch into another.

★ INTERVIEW FAVORITE

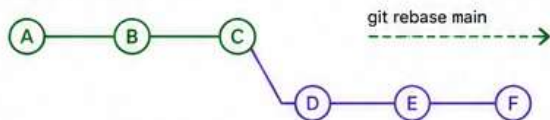
git rebase

Reapplies your commits on top of the target branch.
Creates a linear history.

History with Rebase (Linear)

○ main ○ feature

Before rebase



After rebase



Feature commits are moved to the latest of main.
No merge commit created.

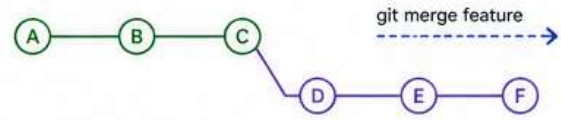
git merge

Combines branches by creating a merge commit.
Preserves the history and branching.

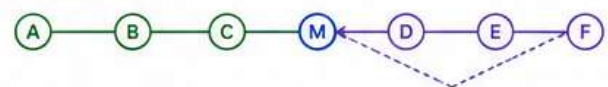
History with Merge (Non-Linear)

○ main ○ feature

Before merge



After merge



A merge commit (M) combines both histories.
History of both branches is preserved.

Key Differences	git rebase	git merge
History	Linear	Non-linear (with merge commits)
How it works	Replays your commits on top	Creates a new merge commit
Commit IDs	New commit IDs (history is rewritten)	Original commit IDs preserved
Best for	Clean history, personal branches	Shared branches, preserving context
Risk	Rewriting history (careful on shared branches)	Safe (no history rewrite)
Command	<code>git rebase main</code>	<code>git merge feature</code>

⚡ When to Use Rebase

- You want a clean, linear project history.
- You're working on a feature branch.
- You haven't pushed your commits yet.

Example

```
git checkout feature
git rebase main
```

💡 When to Use Merge

- You want to preserve the full history.
- You're working on shared branches.
- You've already pushed your commits.

Example

```
git checkout main
git merge feature
```



Interview Tip

Rebase = Reapply commits on top → Linear history
Merge = Combine histories → Non-linear history with merge commit



Branching Strategies

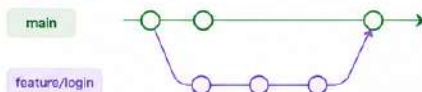
Different approaches for organizing work in Git.



1. Feature Branch

Create a branch for a new feature or fix. Merge it back to main when ready.

How it looks



Best for

Small to medium teams, simple workflows, and isolated features.

Pros

- Simple and easy to understand
- Encourages isolated work
- Low overhead

Example

```
git checkout -b feature/login
# work...
git checkout main
git merge feature/login
```



2. Gitflow

Use dedicated branches for features, releases, and hotfixes. Structured and explicit.

How it looks



Best for

Large projects, multiple teams, scheduled releases.

Pros

- Clear structure and roles
- Good for release management
- Supports parallel work

Example

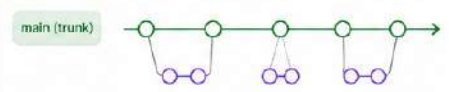
```
git flow feature start login
# work...
git flow feature finish login
```



3. Trunk-based Development

Develop on short-lived branches from main (trunk) and merge back frequently.

How it looks



Best for

High-velocity teams, CI/CD, and continuous delivery.

Pros

- Fast feedback and delivery
- Less merging pain
- Encourages small, frequent commits

Example

```
git checkout -b feat/login
# small changes, commit
git checkout main
git merge --no-ff feat/login
```



Tip: There's no one-size-fits-all. Choose the strategy that matches your team size, release cycle, and tooling.

Best Practices

Build better habits. Write better code. Work better together.



1 Meaningful Commit Messages

Write clear, concise commit messages that explain the “why”, not just the “what”.

CHECKLIST

- ☒ Use the imperative mood (Add, Fix, Update).
- ☒ Keep the subject line under 72 characters.
- ☒ Explain the what and why, not how.
- ☒ Reference issues when applicable.
- ☒ Avoid vague messages like “fix bug”.

GOOD EXAMPLES

```
feat(auth): add login with Google
fix(api): handle null response in user service
docs(readme): update installation steps
refactor(cart): simplify price calculation
test(user): add unit tests for validation
```

AVOID

```
update stuff
fix bug
changes
stuff
```



2 Resolving Conflicts

Conflicts happen. Resolve them carefully and communicate clearly.

CHECKLIST

- ☒ Pull latest changes before starting work.
- ☒ When conflicts occur, understand both sides.
- ☒ Edit files to resolve conflicts.
- ☒ Test to ensure everything works.
- ☒ Commit the resolution with a clear message.

BEST PRACTICES

- Stay calm and read the conflict markers.
- Choose the right changes, don't prefer one side blindly.
- Ask for help if unsure.
- Communicate in your team/channel.

CONFLICT MARKERS (IN FILES)

```
<<<<<< HEAD
// Your changes
=====
// Incoming changes
>>>>>> feature-branch
```



3 Code Reviews

Review code to catch issues early and share knowledge.

CHECKLIST

- ☒ Keep PRs focused and small.
- ☒ Provide a clear description of the changes.
- ☒ Review respectfully and constructively.
- ☒ Test the changes locally if possible.
- ☒ Address feedback and update the PR.

BEST PRACTICES

- Focus on the code, not the person.
- Look for correctness, clarity, and simplicity.
- Use comments to start discussions.
- Approve when it's good to go! 🚀



Remember

Good habits in Git lead to clean history, fewer bugs, and happier teams.



GitHub Features

Powerful built-in tools to automate, deploy, and protect your projects.



1 Actions

Automate your software workflows with CI/CD.

- ✓ Build, test, and deploy automatically
- ✓ Trigger workflows on events (push, PR, etc.)
- ✓ Supports Linux, macOS, Windows
- ✓ Use prebuilt actions or create your own

```
.github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm install
      - run: npm test
```

i Actions help you save time and ship with confidence.



2 Pages

Host static websites directly from your repositories.

- ✓ Publish personal, project, or docs sites
- ✓ Uses GitHub's global CDN
- ✓ Custom domains and HTTPS included
- ✓ Easy setup with a few clicks

<https://username.github.io/repo>



i Great for project sites, documentation, portfolios, and more.



3 Secrets

Store sensitive information securely in your repositories.

- ✓ Keep tokens, API keys, and passwords safe
- ✓ Encrypted at rest and in transit
- ✓ Accessed only in workflows or Actions
- ✓ Manage at repo, org, or environment level

Settings > Secrets and variables > Actions

Repository secrets

API_TOKEN			
DATABASE_URL			
SLACK_WEBHOOK_URL			

i Use secrets—never hardcode sensitive data in your code.



Build better with GitHub

Automate workflows, deploy with ease, and keep secrets secure.
Ship faster. Collaborate better.



Project Essentials

Three must-have files for every repository.



1 README.md

The first thing people see.
Explain what your project is,
why it exists, and how to use it.

Include:

- ✓ Project title and description
- ✓ Features
- ✓ Installation
- ✓ Usage
- ✓ Contributing
- ✓ License

README.md (Example)

Awesome Project 🚀

A short description of what this project does and why it's useful.

🌟 Features

- Fast
- Simple
- Extensible

🔧 Installation

```
git clone https://github.com/username/awesome-project.git
cd awesome-project
```

▶ Usage

```
npm install
npm start
```

👉 Contributing

Contributions are welcome! Please open an issue or submit a pull request.

📄 License

This project is licensed under the MIT License.



2 Licensing

A license tells others how they
can use, share, and contribute
to your project.

Best practices:

- ✓ Choose an appropriate license
- ✓ Add a LICENSE file
- ✓ Mention it in your README

Popular Licenses

MIT	Permissive. Use almost anywhere.
Apache-2.0	Permissive with patent protection.
GPL-3.0	Copyleft. Derivatives must be open source.
BSD-3-Clause	Permissive. Very similar to MIT.

i If you're not sure, MIT is a great default for open source projects.



3 .gitignore

Tells Git which files and
directories to ignore.
Keeps your repo clean.

Common things to ignore:

- ✓ Dependencies (node_modules/)
- ✓ Build outputs (dist/, build/)
- ✓ Environment files (.env)
- ✓ OS / editor files (.DS_Store, *.log, *.vscode/)

.gitignore (Example for Node.js)

```
# Dependencies
node_modules/

# Build output
dist/
build/

# Environment
.env

# Logs
*.log/

*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# OS / Editor
.DS_Store
.vscode/
*.swp
```



Quick Tips

- ✓ Keep your README updated.
It's your project's front door.
- ✓ Pick a license early.
It's easier than changing later.
- ✓ Review your .gitignore
before every commit.

Start Building. Start Committing. Start Collaborating.

Great code starts with good habits.
Keep learning. Keep shipping. Keep growing.



♥ Thanks for being part of the open source community.
Together, we build **better software**.